

LaTune: Lightweight and Adaptive Configuration Tuning for LLM Inference on Edge Devices

Siqi Zhong
Institute for Artificial Intelligence, Peking University
Beijing, China
siqizhong25@stu.pku.edu.cn

Mugeng Liu
School of Computer Science, Peking University
Beijing, China
lmg@pku.edu.cn

Haiyang Shen
Institute for Artificial Intelligence, Peking University
Beijing, China
hyshen@stu.pku.edu.cn

Chongyang Pan
School of Computer Science, Peking University
Beijing, China
pancy2021@stu.pku.edu.cn

Yun Ma*
Institute for Artificial Intelligence, Peking University
Beijing, China
mayun@pku.edu.cn

Abstract

Large Language Models (LLMs) are increasingly deployed on edge devices to address privacy and latency concerns in modern Web applications. While numerous studies focus on inference frameworks, the critical problem of tuning runtime configurations remains largely underexplored. This endeavor is particularly challenging on edge devices due to severe budget limitations and the dynamic variability of system resources.

To address these challenges, we draw upon key insights regarding parameter sensitivity, configuration transferability, and rank stability to propose LaTune, a lightweight and adaptive tuning framework. LaTune is designed to efficiently find optimal runtime configurations by incorporating three complementary components: parameter selection to focus on the most impactful parameters, knowledge transfer to leverage historical data for accelerated search, and two-stage optimization to dynamically select the best configuration based on real-time resource constraints. Experiments across four edge devices and LLMs show that LaTune achieves up to 3.93× higher hypervolume and 6.90× throughput gains over baselines. It accelerates tuning efficiency by 2–3×, converging within 10–20 iterations, and ensures robust execution under heavy contention where static methods fail. Our code is already open-sourced at https://github.com/pkuaiweb/LaTune.

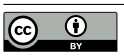
CCS Concepts

• **Software and its engineering** → **Search-based software engineering**; • **Computing methodologies** → *Artificial intelligence.*

Keywords

Large Language Models (LLMs), Edge Inference, Inference Optimization, Adaptive Configuration Tuning

*Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License.
WWW '26, Dubai, United Arab Emirates
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
https://doi.org/10.1145/3774904.3792382

ACM Reference Format:
Siqi Zhong, Mugeng Liu, Haiyang Shen, Chongyang Pan, and Yun Ma. 2026. LaTune: Lightweight and Adaptive Configuration Tuning for LLM Inference on Edge Devices. In *Proceedings of the ACM Web Conference 2026 (WWW '26), April 13–17, 2026, Dubai, United Arab Emirates.* ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/3774904.3792382

Resource Availability:
The source code of this paper has been made publicly available at https://doi.org/10.5281/zenodo.18346520 and the GitHub repository https://github.com/pkuaiweb/LaTune.

1 Introduction

Large Language Models (LLMs) are increasingly embedded in modern Web applications, driving services such as personalized search, recommendation, and interactive content creation [14, 30, 39, 49, 62]. However, the high computational demand of LLM inference means that most systems still rely on cloud-based deployment. This design enables efficient processing but also requires transmitting large volumes of user data to remote servers, raising concerns about privacy, latency, and network dependency. Recently, there has been a growing shift toward running LLMs on edge devices [37, 63] such as personal computers, edge servers, and IoT nodes. With lightweight edge inference engines like llama.cpp [1], models can now run locally on consumer-grade hardware, enabling privacy-preserving and low-latency Web intelligence without full reliance on the cloud.

To ensure a smooth user experience, it is equally important that local inference runs efficiently. Modern inference engines incorporate a variety of optimization techniques, including continuous batching [16, 51, 65], flash attention [17, 18, 46], and chunked prefill [4, 5, 76], to accelerate inference. At the same time, runtime parameters such as batch size and thread count also have a strong impact on performance [7, 41, 66]. Together, the selection of optimization techniques and the tuning of runtime parameters define the *runtime configuration* of the inference engine, which plays a decisive role in overall performance. A desirable configuration should achieve high throughput, i.e., a faster token generation rate during on-device inference, while keeping resource consumption such as GPU memory and CPU usage within acceptable limits. However,

there is no single configuration that remains optimal across devices since edge devices differ widely in architecture and capacity [57]. Therefore, adaptive tuning is essential for achieving efficient and stable inference on diverse edge devices.

However, tuning configurations on resource-constrained devices presents two unique challenges. The first is the limited budget [63] for tuning. Each evaluation involves costly operations such as model loading, decoding, and monitoring, which compete with user tasks for computation. On devices with limited CPU capacity, a tuning process that runs too long may disrupt normal usage or even be terminated by the operating system. The second challenge lies in the variability of device resources [54, 64, 72]. Edge devices are often affected by background processes, causing fluctuations in available memory and compute capacity. As a result, the feasible configuration space is dynamic, and settings that perform well in one run may degrade quickly under different resource conditions.

To address these challenges, we propose **LaTune**, a lightweight adaptive tuning framework that efficiently identifies optimal configurations under tight budgets while maximizing runtime throughput. Our design leverages three key insights. *First*, as only a small subset of parameters dominates performance, we restrict the search to these high-impact factors to minimize tuning costs. *Second*, recognizing that high-performing configurations often generalize across tasks, we reuse historical data to accelerate the tuning process. *Third*, the stability of configuration rankings under resource fluctuations allows us to utilize configurations learned under stable conditions to guide dynamic runtime selection.

Building on these insights, LaTune adopts a simple yet effective strategy that includes three complementary components. To cope with the limited tuning budget, LaTune first performs *parameter selection*, identifying the parameters that have the greatest impact on throughput and resource use, thereby reducing search complexity and improving efficiency. To mitigate the cost of exploring from scratch, LaTune further applies *knowledge transfer*, which reuses high-performing configurations from previous tuning tasks as warm-start candidates under similar workload and hardware conditions. Finally, to handle fluctuating resources, LaTune introduces *two-stage optimization*, where the system learns the Pareto front under stable conditions and selects suitable configurations at runtime according to the current resource state. Together, these components enable LaTune to balance throughput and resource utilization while maintaining robustness across diverse and dynamic edge environments.

We conduct extensive experiments to evaluate LaTune across diverse settings, covering four LLMs with distinct architectures and quantization levels deployed on four heterogeneous edge devices. To rigorously test adaptability, we simulate three distinct resource levels (Low, Medium, High) and compare LaTune against four representative baselines: Default, GA [24], ResTune [70], and SCOOT [15]. Experimental results demonstrate that LaTune consistently outperforms all baselines across varying devices and resource conditions. Specifically, it achieves 1.47×–3.93× higher Hypervolume compared to default settings and surpasses the strongest baselines by 1.05×–6.90× in throughput. Notably, LaTune exhibits superior robustness, maintaining stable performance under high-load scenarios where static configurations fail completely. In terms of efficiency, it identifies high-quality Pareto fronts within 10–20 iterations, converging 2–3× faster than prior methods. Further ablation studies confirm the essential contributions of parameter selection, knowledge transfer, and two-stage optimization to these gains.

In summary, the main contributions of this paper are as follows:

- To the best of our knowledge, this is the first work that systematically studies configuration tuning for LLM inference engines on edge devices, addressing the challenges of resource-constrained and dynamic environments.
- We propose LaTune, a lightweight and adaptive tuning method that incorporates practical design optimizations to make on-device tuning efficient and responsive to changing resources.
- We conduct extensive experiments across diverse models, hardware platforms, and resource settings, demonstrating that LaTune finds better configurations with fewer evaluations, and achieves higher throughput and robustness than existing methods.

2 Background and Motivation

We first review the architecture of on-device LLM inference engines and the challenges inherent in configuring them. We then present a measurement study yielding empirical insights that guide the design of LaTune .

2.1 On-Device LLM Inference Engines

Running LLM inference directly on edge devices provides an effective alternative to cloud-based deployment for Web applications [63]. Processing queries locally avoids data transmission, enhancing privacy and reducing latency for Web services. It enables continuous operation even in bandwidth-limited or offline environments. This capability enhances the responsiveness and reliability of edge-intelligent applications that require real-time interaction.

To support such deployment, several inference engines have been developed to execute LLMs efficiently on local hardware. Representative examples include llama.cpp [1], MLC-LLM [53], and TensorRT-LLM [3]. llama.cpp is a lightweight C/C++ implementation that enables quantized model inference on CPUs and GPUs, serving as the backend for local Web frameworks such as Ollama [2]. MLC-LLM extends machine-learning compilation to support unified deployment across heterogeneous hardware, while TensorRT-LLM integrates NVIDIA's TensorRT library to deliver optimized GPU performance. In contrast, cloud-oriented systems such as vLLM [29] and SGLang [75] are designed for large-scale GPU clusters and therefore fall outside the on-device inference scope of this work.

Despite differences in implementation details such as memory management and task scheduling [42], modern inference engines are commonly organized around the four domains of model preparation, inference execution, system management, and service management [32, 42, 58]. During inference, they expose a set of tunable parameters that jointly affect throughput and latency [15]. As summarized in Appendix Table 2, these parameters can be organized into four hierarchical levels according to their functional scope. In this work, we focus on optimizing runtime-level parameters that directly influence performance while keeping model quality and service semantics unchanged. Specifically, we exclude model-compression options (e.g., quantization precision) and service-layer configurations that are irrelevant to inference efficiency.

2.2 Measurement Study

On-device inference operates under highly dynamic environments and diverse workloads, making universal tuning strategies difficult to design. To understand how configuration parameters interact with varying hardware and resource conditions, we conduct a comprehensive measurement study that provides empirical insights to guide adaptive tuning.

Experimental Setup. We conduct measurements using `llama.cpp` [1] on two representative edge devices: a PC with an RTX 3060 and a Mac with an Apple M4. We evaluate *Qwen3-4B-Q4* and *Phi-Mini-MoE-Q8*, which differ significantly in architecture. To cover diverse workloads, we employ short conversational queries and long-document summarization [47]. Furthermore, we simulate four levels of system load by applying controlled stress to mimic varying resource availability. With the context length fixed at 1024, we measure average throughput (tokens per second, TPS) and VRAM usage.

Insight 1: Uneven parameter contribution. A major challenge in on-device tuning is the large number of exposed parameters, many of which have negligible impact on performance. We conduct a one-factor-at-a-time (OFAT) sensitivity analysis [44] using the default setup as baseline and vary six representative parameters individually. As shown in Figure 1, the impact is highly uneven: increasing `gpu-layers` improves TPS by 43.4% with minimal GPU overhead, while disabling `kv-offload` decreases it by 15.5%. Other parameters, such as `parallel` and `no-cont-batching`, show little effect. Such findings highlight that only a few key parameters dominate efficiency, motivating a focused search on influential factors rather than uniform exploration.

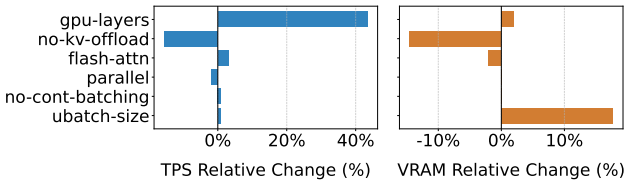


Figure 1: Single-parameter sensitivity of TPS and VRAM usage. Both versus the default value. Each bar shows the percent change from toggling one parameter while holding others fixed (positive = higher TPS / more VRAM).

Insight 2: Transferability of Optimal Configurations. Another difficulty lies in the cost of tuning from scratch for each device or workload. To examine reusability, we construct five tuning tasks that differ in hardware, model, and workload. Each empirically optimal configuration is tested across other tasks. As shown in Figure 2, several configurations perform well beyond their original setting, e.g., *conf.B* even surpasses local optima on tasks A and C, whereas some fail under tighter resource limits. It suggests that high-performing configurations can be transferred across similar tasks to warm-start new tuning processes, but effective reuse requires awareness of task similarity.

Insight 3: Rank Stability Across Resource Conditions. End-device resources fluctuate due to background activities, making static configurations unreliable. We simulate four levels of system load (idle, low, medium, high) and evaluate five configurations under each. Figure 3 shows that the relative TPS order among valid configurations remains stable, yet higher load reduces TPS and can cause some configurations to fail because the available resources are insufficient. This stability indicates that statically identified configurations can effectively guide runtime tuning, reducing repeated exploration when resource conditions change.

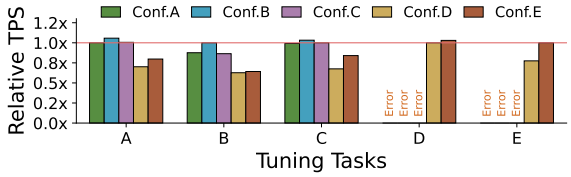


Figure 2: Relative TPS of configurations across tasks. Bars show the relative TPS of Conf.A–E; the red line is the per-task 1.0× baseline. "Error" marks configurations that failed.

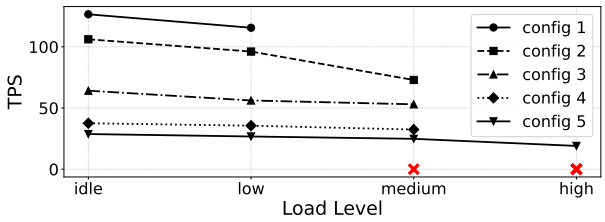


Figure 3: TPS of configurations across load levels. The x-axis lists load (idle to high); the y-axis is TPS. Each line is a configuration; red "×" marks configurations that failed.

3 Problem Formulation

Efficient on-device LLM inference requires selecting proper runtime configurations that balance throughput and resource use under changing hardware conditions. However, these configurations depend on multiple interacting parameters and a dynamic resource environment, making the tuning process a continuous optimization problem rather than a one-time search.

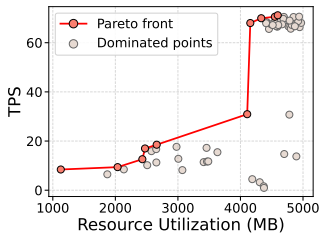
We formalize the tuning problem as follows. Let the space of all tunable parameters be $\Theta = \Theta_1 \times \Theta_2 \times \cdots \times \Theta_m$, where Θ_i represents the domain of the i-th parameter exposed by the inference engine. A configuration $\mathbf{P} = (p_1, p_2, \ldots, p_m)$ is a point in this space, where each $p_i \in \Theta_i$ corresponds to a specific setting of parameter i. The inference performance of $\mathbf{P}$ can be characterized by two objectives: throughput $T(\mathbf{P})$ and resource utilization $U(\mathbf{P})$. Throughput measures the token generation efficiency (TPS), while resource utilization reflects the fraction of available hardware resources consumed by inference.

The goal is to maximize throughput while minimizing resource usage. Under a fixed resource condition, this can be expressed as a bi-objective optimization problem:

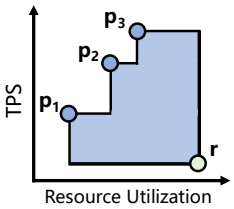
$$\max_{\mathbf{P}\in\Theta} \big(T(\mathbf{P}), -U(\mathbf{P})\big). \tag{1}$$

Solving this problem yields a set of non-dominated configurations forming the *Pareto front*, where improving one objective (e.g., throughput) inevitably degrades the other (e.g., resource use). Each

point on this front represents an optimal trade-off between performance and efficiency, as illustrated in Figure 4a, and the overall quality of the front can be quantified by the *hypervolume* indicator, as illustrated in Figure 4b.



(a) The red curve marks non-dominated configurations; lighter points are dominated (worse TPS and/or higher resource use).



(b) The shaded region represents the hypervolume dominated by the Pareto-optimal relative to a reference point.

Figure 4: Pareto-optimal configs and corresponding hypervolume. (a) Pareto front shows trade-offs between TPS and resource usage. (b) Hypervolume measures the extent of the objective space dominated by the Pareto front.

In practice, end-device resources fluctuate over time due to competing background processes and thermal constraints. These variations dynamically reshape the feasible configuration space, causing previously optimal settings to become suboptimal or infeasible. To capture this behavior, we introduce a time-dependent resource constraint R_t, representing available system capacity at time t. The tuning objective is thus refined to maximize throughput within the current resource limit:

$$\max_{\mathbf{P}\in\Theta} \; T(\mathbf{P}) \quad \text{s.t.} \quad U(\mathbf{P}) \le R_t. \tag{2}$$

4 LaTune

The goal of LaTune is to address the tuning problem by selecting runtime configurations that maximize throughput while adapting to changing resource constraints on edge devices. Guided by the three key insights, LaTune integrates focused search with the reuse of prior results and lightweight runtime adaptation.

4.1 Overview

The workflow of LaTune is illustrated in Figure 5. First, ① historical observations are retrieved from the *Historical Repository* to support the *Parameter Selector*. Meanwhile, ② the meta-features of both historical tasks and the current task (provided by the inference instance) are delivered to the *Knowledge Adapter*, which identifies similar tasks and retrieves their optimal configurations. ③ These transferred configurations are aligned with the current tunable parameter set to construct candidate configurations, and ④ the resulting initial points are then passed to the *Evaluator* for performance assessment. The *Evaluator* executes point initial points on the inference instance ⑤, which returns runtime metrics ⑥. These observations are used to update the *Surrogate Model* ⑦, whose predicted mean and variance ⑧ are passed to the *Acquisition Function* to guide exploration. New configurations proposed by the acquisition function ⑨ are fed back to the evaluator, forming an iterative optimization loop. As tuning proceeds, all evaluated observations are aggregated by the *Pareto Frontier* ⑩ to build the frontier of optimal trade-offs. The resulting *Optimal Set* ⑪ is then passed to the *Resource-aware Selector*, which finally determines ⑫ the optimal configuration to deploy on the inference engine.

4.2 Parameter Selection

High dimensionality in the configuration space challenges on-device tuning. Guided by Insight 1, LaTune tackles this by leveraging historical data to build a surrogate model and identifying influential parameters via Shapley-value analysis.

LaTune trains a lightweight *Random Forest surrogate* [25] to capture complex, non-linear configuration-performance relationships. Random Forest is selected for its robustness to noise and ability to model parameter interactions. Formally, it approximates the mapping from configurations to multi-objective metrics:

$$\hat{f} : \mathbf{P} \mapsto \mathbf{Y}, \quad \mathbf{Y} = (y_{\text{tps}}, y_{\text{gpu}}), \tag{3}$$

where $\mathbf{P} = (p_1, p_2, \ldots, p_m)$ represents a configuration vector and $\mathbf{Y}$ denotes the corresponding throughput and resource utilization.

However, simply modeling the output is insufficient because parameter effects are typically interdependent. Traditional sensitivity analyses, such as OFAT [44] or correlation-based methods [23], often fail to capture these intricate interactions. To overcome this limitation, LaTune employs Shapley values [40] to fairly attribute the marginal contribution of each parameter. For each objective y_k, the Shapley value $\phi_j^{(k)}$ of parameter p_j is computed as:

$$\phi_j^{(k)} = \sum_{S\subseteq P\setminus\{j\}} \frac{|S|!(|P|-|S|-1)!}{|P|!} \left[\hat{f}_{S\cup\{j\}}^{(k)} - \hat{f}_S^{(k)}\right], \tag{4}$$

where S represents parameter subsets excluding p_j, and $\hat{f}_S^{(k)}$ is the expected metric output. A larger $\phi_j^{(k)}$ indicates that p_j contributes more to the objective.

Given that throughput and resource usage differ in both scale and priority, LaTune aggregates the contributions across objectives using a weighted scheme:

$$\Phi_j = \sum_{k=1}^{K} w_k \cdot \tilde{\phi}_j^{(k)}, \tag{5}$$

where $\tilde{\phi}_j^{(k)}$ is the normalized Shapley value and $\mathbf{w}$ represents objective weights. We finally prune the search space by selecting the top-N parameters with the highest importance Φ_j to form the active subset:

$$\mathcal{S}_{\text{sel}} = \{p_{j_1}, p_{j_2}, \ldots, p_{j_N}\}. \tag{6}$$

4.3 Knowledge Transfer

Starting each tuning task from scratch is costly and inefficient, especially on resource-constrained devices. Based on Insight 2, LaTune introduces knowledge transfer to warm-start new tuning tasks with experience from previous ones.

Each historical tuning task is represented by a *meta-feature vector* $\mathbf{m} \in \mathbb{R}^F$ that summarizes the characteristics of the model, hardware, and workload. This vector $\mathbf{m}$ comprises model specifications (e.g., parameter scale, quantization precision), hardware resource metrics (CPU/GPU cores, memory capacity), and workload patterns (input

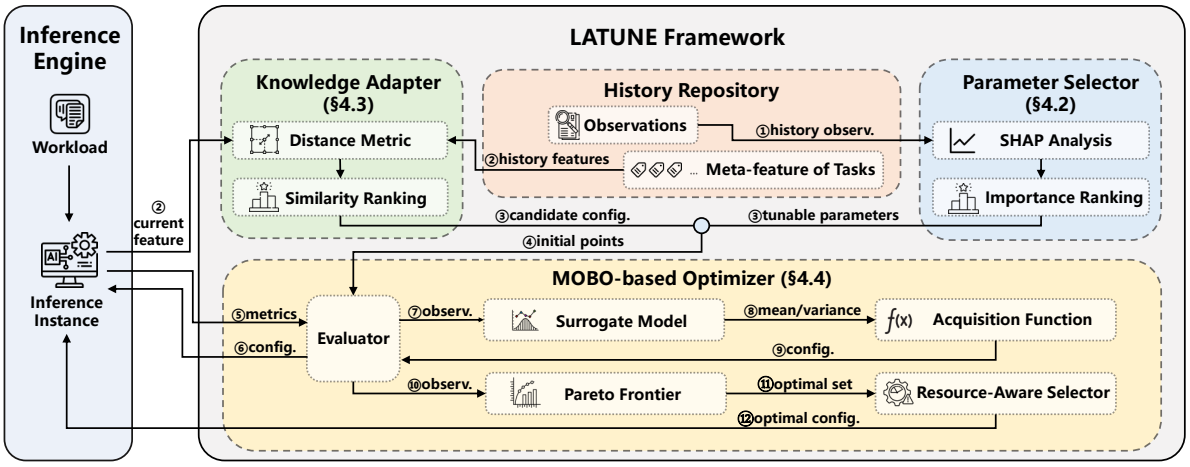


Figure 5: Workflow for adaptive tuning in local LLM inference. The left shows the inference engine, while the right presents LaTune, comprising three modules: Parameter Selector, Knowledge Adapter, and MOBO-based Optimizer (Sections 4.2–4.4).

length, request concurrency), with a full specification detailed in Appendix A.2. Each record in the historical repository is denoted as $\mathcal{H}_i = (\mathbf{m}_i, C_i^\star)$, where $C_i^\star$ is the set of optimal configurations found in task i.

To identify similar tasks, LaTune computes a kernel-weighted similarity measure:

$$g_i = K\left(\frac{\|\mathbf{m}_i - \mathbf{m}_{T+1}\|_2}{h}\right), \quad K(t) = \begin{cases} \frac{3}{4}(1-t^2), & |t| \le 1, \\ 0, & \text{otherwise.} \end{cases} \tag{7}$$

Here h controls how localized the transfer is, and $K(\cdot)$ is the Epanechnikov kernel that emphasizes nearby tasks. LaTune then selects the top-K most similar tasks based on g_i and extracts their best configurations as warm-start candidates. Shared parameters are directly reused, while new ones are initialized randomly within valid ranges.

4.4 Two-Stage Optimization

Edge resource fluctuations caused by background activities or thermal limits often render static configurations suboptimal or infeasible. Guided by Insight 3, which reveals that configuration rankings remain stable despite such volatility, LaTune adopts a two-stage optimization strategy. Specifically, the offline optimization stage utilizes Multi-Objective Bayesian Optimization (MOBO) to construct a diverse Pareto front of trade-offs, serving as the candidate pool for the online adaptation stage, which dynamically selects the throughput-maximizing configuration satisfying dynamic resource constraints.

4.4.1 Optimal Set Construction via MOBO. In the offline optimization stage, LaTune employs a *multi-objective Bayesian optimization* (MOBO) framework [28] to construct a diverse, high-quality configuration set under stable conditions. MOBO is well-suited for this scenario as it models expensive evaluations (e.g., model loading) through a probabilistic surrogate [11], efficiently guiding the exploration of configurations that jointly maximize throughput and minimize resource usage.

Surrogate Model. MOBO relies on a surrogate model to approximate the expensive black-box objective functions. In LaTune, this surrogate is implemented using *Gaussian Process Regression* (GPR) [45], which models each objective function $f(\mathbf{P})$ as a Gaussian distribution parameterized by a mean $\mu(\mathbf{P})$ and variance $\sigma^2(\mathbf{P})$. Given a configuration $\mathbf{P}$, these parameters are estimated from previously evaluated samples, providing both a prediction and an uncertainty estimate. To balance smoothness and flexibility, LaTune uses the Matérn kernel as the covariance function. Separate GPR models are trained for throughput and resource consumption to prevent interference caused by different scales and correlations.

Acquisition Function. Once the surrogate model estimates the objective landscape, the next question is which configuration should be evaluated next. To guide this exploration, LaTune adopts the *Expected Hypervolume Improvement* (EHVI) acquisition function, which quantifies how much a new candidate can expand the dominated region of the current Pareto front. Formally, the EHVI is:

$$\text{EHVI}(\mathbf{x}) = \mathbb{E}\left[\max\left(0, \text{HV}(Y \cup \{f(\mathbf{x})\}) - \text{HV}(Y)\right)\right], \tag{8}$$

where $\text{HV}(\cdot)$ denotes the hypervolume of the dominated region in the objective space, Y is the set of evaluated configurations, and $f(\mathbf{x})$ is the predicted objective vector for candidate $\mathbf{x}$. Intuitively, configurations with higher EHVI values are more likely to improve the trade-off front, balancing exploration of uncertain regions and exploitation of known promising areas.

Pareto-oriented Search. The MOBO process iteratively applies this surrogate–acquisition cycle to approximate the Pareto front of configurations. Each iteration selects the most promising candidate based on EHVI, evaluates it on the actual inference engine, and updates the GPR models with the new observation. After MOBO procedure, LaTune obtains a Pareto-optimal configuration set that represents diverse trade-offs between throughput and resource utilization. This front serves as a candidate pool for fast online adaptation without repeating the heavy optimization process.

Table 1: Hypervolume (HV, %) comparison across different devices. Rows list tuning methods evaluated on four models. Within each model, the four shaded subcolumns denote diverse devices from dark to light. Bold indicates the best and underline the second best per column; "N/A" means the model was infeasible on that device due to excessive resource consumption.

RTX 4090 RTX 3060 Apple M4 Orin-Nano

Method	Qwen3-4B-Q4				Qwen3-4B-Q8				Phi-Mini-MoE-Q4				Phi-Mini-MoE-Q8			
Default	24.75	25.57	65.77	23.64	16.32	20.38	56.88	19.54	21.94	17.61	39.57	20.55	26.16	16.25	22.66	N/A
GA	59.43	72.25	72.98	52.86	52.62	56.64	70.75	66.95	50.06	42.30	62.92	49.51	65.24	41.65	<u>73.62</u>	N/A
ResTune	<u>65.27</u>	<u>73.55</u>	83.02	<u>55.90</u>	54.86	58.54	<u>82.10</u>	<u>70.81</u>	<u>53.10</u>	<u>51.18</u>	<u>69.82</u>	<u>56.96</u>	67.68	<u>45.72</u>	72.30	N/A
SCOOT	65.11	73.00	<u>87.58</u>	45.40	<u>56.67</u>	<u>60.86</u>	78.60	67.11	52.53	47.88	64.33	49.51	<u>67.80</u>	44.91	67.10	N/A
LaTune	**70.23**	**76.34**	**96.50**	**64.99**	**60.16**	**64.13**	**86.60**	**76.82**	**58.53**	**54.89**	**78.25**	**66.04**	**70.78**	**50.52**	**83.51**	N/A

4.4.2 Runtime Resource-aware Configuration Selection. While the offline optimization stage establishes the Pareto front under representative conditions, runtime resources fluctuate rapidly due to competing applications. To ensure responsiveness, the online adaptation stage executes a lightweight selection from the precomputed Pareto set based on real-time resource availability.

Let R_t denote the available resource budget at time t. Given R_t, the goal is to select the configuration that maximizes throughput while satisfying the resource constraint:

$$\mathbf{P}_t^\star = \arg\max_{\mathbf{P}\in C_{\text{pareto}}} T(\mathbf{P}) \quad \text{s.t.} \quad U(\mathbf{P}) \le R_t, \tag{9}$$

where C_{pareto} denotes the Pareto-optimal configuration set obtained during MOBO, $T(\mathbf{P})$ is the empirically observed throughput for configuration $\mathbf{P}$, and $U(\mathbf{P})$ is its corresponding resource utilization. Since this process only evaluates the currently available resources, its computational overhead is negligible, allowing dynamic adjustment to resource changes.

5 Experiments

We evaluate the performance of LaTune by addressing the following research questions:

- **RQ1 (Overall Performance):** How does the configuration quality of LaTune compare to state-of-the-art tuning methods?
- **RQ2 (Tuning Efficiency):** How efficiently can LaTune identify high-quality solutions, especially under the limited computational budget of edge devices?
- **RQ3 (Ablation Study):** How do the individual components, including parameter selection, knowledge transfer, and two-stage optimization, contribute to the system's effectiveness?

5.1 Experimental Setup

Implementation and Environment. We implement LaTune using approximately 2.8K lines of code. The backend is `llama.cpp` [1], though the design is engine-agnostic. Experiments are conducted on diverse representative edge platforms: an NVIDIA RTX 4090 (24 GB VRAM) for high-end, an NVIDIA RTX 3060 (12 GB VRAM) for mainstream, a Mac mini with M4 chip (16 GB memory) for Metal-based execution, and a Jetson Orin Nano (8 GB) for low-power embedded scenarios.

Models and Workloads. We evaluate four LLMs with different architectures and quantization levels, including Qwen3-4B and Phi-Mini-MoE with INT4 and INT8 quantization, to assess generalization across dense and mixture-of-experts models. Prompts are sampled from the ShareGPT dataset [47] following prior work [20, 34, 43, 68]. We further define low, medium, and high load levels using GPU utilizations of approximately 6.4%, 38.8%, and 69.9%, respectively, induced by controlled background GPU operations.

Baselines. We compare LaTune against four representative alternatives to cover a range of tuning strategies.

- **Default**: The default configuration of the inference engine, representing a no-tuning baseline.
- **Genetic Algorithm (GA)** [24]: A classic heuristic-based method.
- **ResTune** [70]: A state-of-the-art method that uses transfer learning and constrained Bayesian optimization for tuning.
- **SCOOT** [15]: A recent Bayesian optimization framework designed for tuning vLLM in cloud environments.

To ensure the validity of all baselines, we conduct a thorough review and open-source the code for community verification.

Metrics. We evaluate three metrics. **Throughput** is measured in tokens per second (TPS). **Resource usage** is quantified as P95 VRAM consumption on NVIDIA GPUs or P95 RAM usage on unified memory platforms [29, 50, 59]. Although resource usage is not reported explicitly, it is incorporated into the **Hypervolume (HV)** metric, which measures the objective space dominated by the Pareto-optimal configuration set [27, 69]. A larger HV indicates a better trade-off between throughput and resource efficiency. All results are averaged over three independent runs.

5.2 Overall Performance (RQ1)

We evaluate the quality of the final configurations identified by LaTune against baselines, under both static offline settings and dynamic resource constraints.

5.2.1 Quality of Optimal Configuration Sets. Table 1 presents the HV achieved over 50 tuning iterations by each method across all tested devices and models. A higher HV value signifies a superior set of trade-off configurations. The results show that LaTune consistently achieves the highest HV in all scenarios, demonstrating its strong generalization capabilities. Notably, LaTune substantially outperforms the Default method by a large margin, achieving HV values that are approximately 1.47× to 3.93× higher across all evaluated settings.

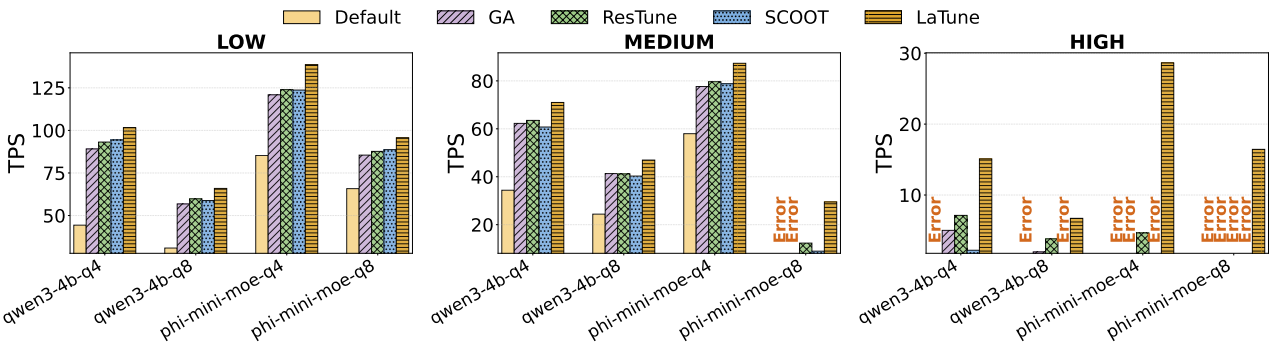


Figure 6: TPS across models and methods at LOW/MEDIUM/HIGH loads. "Error" denotes failures. LaTune achieves the highest TPS and broadens the feasible regime.

This advantage stems from two factors: *knowledge transfer*, which provides strong initial configurations under tight evaluation budgets, and *parameter selection*, which narrows the search to the most impactful parameters and thus explores the Pareto front more effectively. By comparison, the strongest baselines (ResTune/SCOOT) rely on Bayesian optimization and tend to benefit from longer iterative feedback, while GA is more susceptible to local optima, leading to weaker configurations under limited budgets.

5.2.2 Performance under Dynamic Resource Conditions. Figure 6 shows the TPS achieved by each method under three simulated system load levels: Low (abundant resources), Medium (moderate load), and High (heavily constrained resources). The results demonstrate the superior performance and robustness of LaTune. Across all load levels, LaTune consistently achieves higher throughput than all baselines, outperforming the strongest baseline by 1.05×–6.90×.

Under heavy contention, most baselines fail due to infeasible statically chosen configurations, whereas LaTune remains fully operational and maintains stable performance. For instance, on *Phi-Mini-MoE-Q8*, LaTune still sustains 16.4 TPS, while all other methods drop to zero. This robustness stems from LaTune's two-stage optimization, which enables runtime selection of a feasible, high-performance configuration from a pre-computed Pareto-optimal set based on current resource availability.

To ensure robustness, we further validate LaTune under real-world contention scenarios (e.g., video playback, gaming). These results demonstrate that LaTune consistently outperforms baselines, as detailed in Appendix A.3.

5.3 Tuning Efficiency (RQ2)

We evaluate the search efficiency of LaTune by tracking the evolution of the HV metric over 50 iterations, as shown in Figure 7. LaTune consistently outpaces all baselines across different hardware platforms, achieving high-quality Pareto fronts significantly earlier than competitors. Specifically, LaTune rapidly converges, reaching strong HV values within 10–20 evaluations, whereas baselines require 2–3× more iterations to achieve comparable or lower results. This acceleration stems from two key mechanisms: the parameter selector reduces the effective search space by focusing on dominant factors, and knowledge transfer provides high-quality initial points to warm-start the optimization.

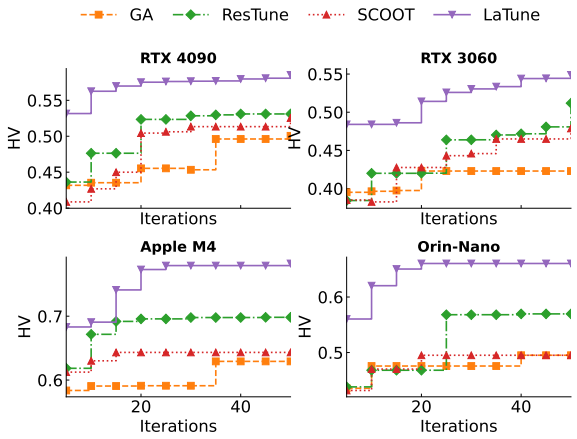


Figure 7: Tuning Efficiency Comparison. Curves plot the HV at each five iterations across methods on four devices. LaTune reaches higher HV sooner and achieves the best final HV across all platforms.

Beyond search efficiency, we validate the system overhead on an RTX 3060. The overhead proves to be lightweight: offline components (parameter analysis and MOBO) require approximately 1.5 s per step, while the online resource-aware selection is virtually instantaneous (≈0.12 s). Furthermore, the historical repository consumes negligible storage (<1 MB), ensuring LaTune fits within the constraints of edge devices.

5.4 Ablation Studies (RQ3)

To isolate the contribution of each component, we evaluate three ablated variants: *w/o-PS* (excludes parameter selection), *w/o-KT* (excludes knowledge transfer), and *w/o-TS* (excludes runtime adaptation).

Impact of PS and KT. Figure 8 illustrates the roles of parameter selection and knowledge transfer in tuning efficiency. The *w/o-PS* variant exhibits sluggish improvement, confirming that exploring the full parameter space is highly inefficient. Meanwhile, *w/o-KT* starts with a significantly lower HV compared to the full model (e.g., 0.41 vs. 0.56 on Jetson Orin). By combining both, LaTune achieves relative improvements of approximately 11% over both variants.

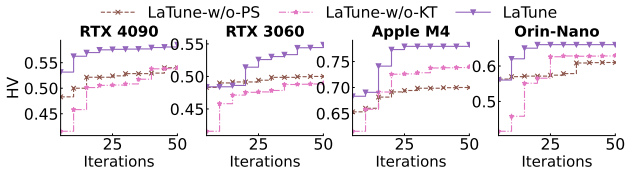


Figure 8: Ablation study on Parameter Selection (PS) and Knowledge Transfer (KT). Removing PS slows down improvement, while removing KT leads to a worse initial performance.

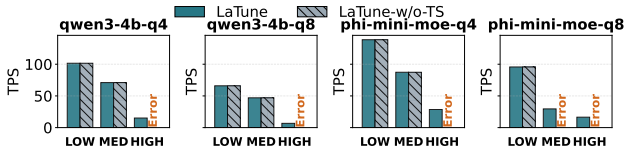


Figure 9: Ablation study of Two-Stage Optimization (TS) under varying system loads. "Error" denotes failures.

These results demonstrate their complementary nature: KT ensures a strong start (warm-start), while PS accelerates the subsequent search.

Impact of Two-Stage Optimization. Figure 9 highlights the necessity of runtime adaptation under dynamic resource conditions. While the static baseline (*w/o-TS*) functions under low system loads, it fails completely under high pressure. For instance, on *Phi-Mini-MoE, w/o-TS* crashes due to resource exhaustion, whereas LaTune successfully sustains 28.6 TPS. This confirms that the online adaptation stage is critical for preventing failures and maintaining performance when edge resources diminish.

6 Related Work

We situate our work within prior research on local LLM systems, efficient inference, and automated configuration tuning.

6.1 Local LLM Inference Engines

Lightweight inference engines have enabled LLMs to run on edge devices and consumer-grade hardware. Representative systems include `llama.cpp` [1], a C++-based engine optimized for CPU and low-level GPU execution, and MLC-LLM [53], a compiler-based framework built on TVM [13] for cross-platform deployment. These engines support low-bit quantization, reduce memory overhead, and provide compatibility across hardware backends such as CPU, CUDA, Metal, and WASM [36, 38, 63]. Compared with cloud-hosted services, local deployment faces tight memory, compute, and power limits. In this setting, inference parameters such as context length, batch size, and GPU layer allocation must be carefully tuned to avoid failures or degraded throughput.

However, existing local engines mostly rely on static configurations or manual tuning. They lack automated mechanisms to adjust runtime parameters under changing system conditions. LaTune addresses this limitation by introducing an adaptive tuning method that automatically selects optimal configurations for local inference engines based on current resource availability.

6.2 Efficient Inference Techniques

To improve LLM inference efficiency, researchers have proposed both model-level and system-level optimizations. On the model side, architectures such as Multi-Query Attention (MQA) [48] and Grouped-Query Attention (GQA) [6, 10] reduce computation and memory cost. Quantization techniques [35, 73] compress models into INT8 or INT4 formats to enable deployment on resource-limited devices with minimal accuracy loss. On the system side, paged attention [29] and speculative decoding [31, 52, 61] reduce memory footprint and latency for long-context generation. Prefix caching [21, 22, 29] and continuous batching [16, 51, 65] improve GPU utilization for multi-turn or multi-user inference.

While these techniques enhance efficiency, performance still depends on proper configuration. Current engines integrate these methods but cannot dynamically adapt to hardware changes. Our work complements this by focusing on configuration-level optimization: instead of modifying models, we automatically adjust parameters to sustain high throughput under fluctuating resources.

6.3 Configuration Tuning

Configuration tuning has been widely studied in domains such as databases [26, 56, 67], distributed frameworks [8, 12, 33], and compilers [9, 13, 74]. Common strategies include Bayesian Optimization [8, 19, 33], Reinforcement Learning [55, 67], and heuristic search [60, 77], which aim to find near-optimal settings under limited evaluation budgets. In LLM systems, SCOOT [15] applies multi-objective Bayesian optimization to cloud workloads, and OnlineTune [71] employs Gaussian Process-based BO for optimizing runtime configurations in deep learning systems.

However, these studies assume static resources, making them unsuitable for dynamic edge environments. To our knowledge, no prior work addresses adaptive tuning for local LLMs under time-varying constraints. We fill this gap by integrating parameter importance analysis, multi-objective Bayesian optimization, and runtime sensing into a unified framework.

7 Conclusion

This paper tackles the critical challenge of optimizing LLM inference on resource-constrained and dynamic edge devices. We introduced LaTune, a lightweight and adaptive tuning framework that efficiently navigates the complex configuration space by focusing on high-impact parameters, transferring knowledge from past tasks, and dynamically adapting to resource fluctuations through a two-stage optimization process. Our extensive experiments confirm that LaTune significantly outperforms existing methods, finding high-quality configurations more quickly while delivering higher throughput and greater robustness, providing a practical solution for efficient LLM deployment in dynamic edge environments.

Acknowledgments

This work was supported by the Beijing Natural Science Foundation (No. L253005), the National Natural Science Foundation of China (No. 62595734), the Key Laboratory of High Confidence Software Technologies (Peking University), the Ministry of Education, and the Center for Data Space Technology and System, Peking University.

References

[1] 2024. Llama.cpp. https://github.com/ggerganov/llama.cpp. Retrieved on October 1, 2025.

[2] 2024. Ollama: Run Large Language Models Locally. https://ollama.com. Retrieved on October 1, 2025.

[3] 2025. TensorRT-LLM: A TensorRT Toolbox for Optimized Large Language Model Inference. https://github.com/NVIDIA/TensorRT-LLM. Retrieved on October 1, 2025.

[4] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming {Throughput-Latency} tradeoff in {LLM} inference with {Sarathi-Serve}. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 117–134.

[5] Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S Gulavani, and Ramachandran Ramjee. 2023. Sarathi: Efficient llm inference by piggybacking decodes with chunked prefills. *arXiv preprint arXiv:2308.16369* (2023).

[6] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebron, and Sumit Sanghai. 2023. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 4895–4901.

[7] Ahsan Ali, Riccardo Pinciroli, Feng Yan, and Evgenia Smirni. 2020. Batch: machine learning inference serving on serverless platforms with adaptive batching. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 69.

[8] Omid Alipourfard, Hongqiang Harry Liu, Jianshu Chen, Shivaram Venkataraman, Minlan Yu, and Ming Zhang. 2017. Cherrypick: adaptively unearthing the best cloud configurations for big data analytics. In *Proceedings of the 14th USENIX Conference on Networked Systems Design and Implementation*. 469–482.

[9] Jason Ansel, Shoaib Kamil, Kalyan Veeramachaneni, Jonathan Ragan-Kelley, Jeffrey Bosboom, Una-May O'Reilly, and Saman Amarasinghe. 2014. OpenTuner: an extensible framework for program autotuning. In *Proceedings of the 23rd International Conference on Parallel Architectures and Compilation*. 303–316.

[10] William Brandon, Mayank Mishra, Aniruddha Nrusimha, Rameswar Panda, and Jonathan Ragan-Kelley. 2024. Reducing Transformer Key-Value Cache Size with Cross-Layer Attention. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.

[11] Eric Brochu, Vlad M Cora, and Nando De Freitas. 2010. A tutorial on Bayesian optimization of expensive cost functions, with application to active user modeling and hierarchical reinforcement learning. *arXiv preprint arXiv:1012.2599* (2010).

[12] Chao Chen, Shixin Huang, Xuehai Qian, and Zhibin Yu. 2025. Swift: fast performance tuning with GAN-generated configurations. In *Proceedings of the 2025 USENIX Conference on Usenix Annual Technical Conference*. 1553–1568.

[13] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. 2018. TVM: An Automated End-to-End Optimizing Compiler for Deep Learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. 578–594.

[14] Zhiyang Chen, Yun Ma, Haiyang Shen, and Mugeng Liu. 2025. WeInfer: Unleashing the Power of WebGPU on LLM Inference in Web Browsers. In *Proceedings of the ACM on Web Conference 2025*. 4264–4273.

[15] Ke Cheng, Zhi Wang, Wen Hu, Tiannuo Yang, Jianguo Li, and Sheng Zhang. 2025. SCOOT: SLO-Oriented Performance Tuning for LLM Inference Engines. In *Proceedings of the ACM on Web Conference 2025*. 829–839.

[16] Seungbeom Choi, Jeonghoe Goo, Eunjoo Jeon, Mingyu Yang, and Minsung Jang. 2025. ELIS: Efficient LLM Iterative Scheduling System with Response Length Predictor. *arXiv preprint arXiv:2505.09142* (2025).

[17] Tri Dao. 2023. Flashattention-2: Faster attention with better parallelism and work partitioning. *arXiv preprint arXiv:2307.08691* (2023).

[18] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In *Advances in Neural Information Processing Systems*. Curran Associates, Inc., 16344–16359.

[19] Songyun Duan, Vamsidhar Thummala, and Shivnath Babu. 2009. Tuning database configuration parameters with iTuned. *Proc. VLDB Endow.* (2009).

[20] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. 2024. Cost-efficient large language model serving for multi-turn conversations with CachedAttention. In *Proceedings of the 2024 USENIX Conference on Usenix Annual Technical Conference*. 111–126.

[21] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. 2024. Cost-Efficient Large Language Model Serving for Multi-turn Conversations with CachedAttention. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*. 111–126.

[22] In Gim, Guojun Chen, Seung-Seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. 2024. Prompt Cache: Modular Attention Reuse for Low-Latency Inference. In *Proceedings of the Seventh Annual Conference on Machine Learning and Systems, MLSys 2024, Santa Clara, CA, USA, May 13-16, 2024*.

[23] Mark A Hall. 1999. *Correlation-based feature selection for machine learning*. Ph. D. Dissertation. The University of Waikato.

[24] Jeffrey Horn, Nicholas Nafpliotis, and David E Goldberg. 1994. A niched Pareto genetic algorithm for multiobjective optimization. In *Proceedings of the first IEEE conference on evolutionary computation. IEEE world congress on computational intelligence*. 82–87.

[25] Frank Hutter, Holger H Hoos, and Kevin Leyton-Brown. 2011. Sequential model-based optimization for general algorithm configuration. In *International conference on learning and intelligent optimization*. Springer, 507–523.

[26] Konstantinos Kanellis, Cong Ding, Brian Kroth, Andreas Müller, Carlo Curino, and Shivaram Venkataraman. 2022. LlamaTune: sample-efficient DBMS configuration tuning. *Proc. VLDB Endow.* (2022).

[27] Florian Karl, Tobias Pielok, Julia Moosbauer, Florian Pfisterer, Stefan Coors, Martin Binder, Lennart Schneider, Janek Thomas, Jakob Richter, Michel Lang, Eduardo C. Garrido-Merchán, Juergen Branke, and Bernd Bischl. 2023. Multi-Objective Hyperparameter Optimization in Machine Learning—An Overview. *ACM Trans. Evol. Learn. Optim.* (2023).

[28] Joshua Knowles. 2006. ParEGO: A hybrid algorithm with on-line landscape approximation for expensive multiobjective optimization problems. *IEEE transactions on evolutionary computation* (2006), 50–66.

[29] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 611–626.

[30] Hanyu Lai, Xiao Liu, Iat Long Iong, Shuntian Yao, Yuxuan Chen, Pengbo Shen, Hao Yu, Hanchen Zhang, Xiaohan Zhang, Yuxiao Dong, and Jie Tang. 2024. AutoWebGLM: A Large Language Model-based Web Navigating Agent. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 5295–5306.

[31] Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning*. 19274–19286.

[32] Baolin Li, Yankai Jiang, Vijay Gadepally, and Devesh Tiwari. 2024. Llm inference serving: Survey of recent advances and opportunities. In *2024 IEEE High Performance Extreme Computing Conference (HPEC)*. IEEE, 1–8.

[33] Yang Li, Huaijun Jiang, Yu Shen, Yide Fang, Xiaofeng Yang, Danqing Huang, Xinyi Zhang, Wentao Zhang, Ce Zhang, Peng Chen, and Bin Cui. 2023. Towards General and Efficient Online Tuning for Spark. *Proc. VLDB Endow.* (2023).

[34] Chaofan Lin, Zhenhua Han, Chengruidong Zhang, Yuqing Yang, Fan Yang, Chen Chen, and Lili Qiu. 2024. Parrot: efficient serving of LLM-based applications with semantic variable. In *Proceedings of the 18th USENIX Conference on Operating Systems Design and Implementation*. 929–945.

[35] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. 2024. AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration. In *Proceedings of Machine Learning and Systems*. 87–100.

[36] Mugeng Liu, Haiyang Shen, Yixuan Zhang, Hong Mei, and Yun Ma. 2025. WebAssembly for container runtime: Are we there yet? *ACM Transactions on Software Engineering and Methodology* (2025), 1–22.

[37] Mugeng Liu, Siqi Zhong, Weichen Bi, Yixuan Zhang, Zhiyang Chen, Zhenpeng Chen, Xuanzhe Liu, and Yun Ma. 2025. A First Look at Bugs in LLM Inference Engines. *arXiv preprint arXiv:2506.09713* (2025).

[38] Mugeng Liu, Siqi Zhong, Qi Yang, Yudong Han, Xuanzhe Liu, and Yun Ma. 2025. WebANNS: Fast and Efficient Approximate Nearest Neighbor Search in Web Browsers. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2483–2492.

[39] Qidong Liu, Xiangyu Zhao, Yuhao Wang, Yejing Wang, Zijian Zhang, Yuqi Sun, Xiang Li, Maolin Wang, Pengyue Jia, Chong Chen, Wei Huang, and Feng Tian. 2025. Large Language Model Enhanced Recommender Systems: Methods, Applications and Trends. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining, V.2, KDD 2025, Toronto ON, Canada, August 3-7, 2025*. 6096–6106.

[40] Scott M Lundberg and Su-In Lee. 2017. A unified approach to interpreting model predictions. *Advances in neural information processing systems* 30 (2017).

[41] Bowen Pang, Kai Li, and Feifan Wang. 2025. Optimizing LLM Inference Throughput via Memory-aware and SLA-constrained Dynamic Batching. *arXiv preprint arXiv:2503.05248* (2025).

[42] Sihyeong Park, Sungryeol Jeon, Chaelyn Lee, Seokhun Jeon, Byung-Soo Kim, and Jemin Lee. 2025. A Survey on Inference Engines for Large Language Models: Perspectives on Optimization and Efficiency. *arXiv preprint arXiv:2505.01658* (2025).

[43] Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2025. Mooncake: Trading More Storage for Less Computation — A KVCache-centric Architecture for Serving LLM Chatbot. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*. 155–170.

[44] Andrea Saltelli, Marco Ratto, Terry Andres, Francesca Campolongo, Jessica Cariboni, Debora Gatelli, Michaela Saisana, and Stefano Tarantola. 2008. *Global Sensitivity Analysis: The Primer*. John Wiley & Sons.

[45] Matthias Seeger. 2004. Gaussian processes for machine learning. *International journal of neural systems* (2004), 69–106.

[46] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. 2024. Flashattention-3: Fast and accurate attention with asynchrony and low-precision. *Advances in Neural Information Processing Systems* 37 (2024), 68658–68685.

[47] ShareAI Lab. 2023. *ShareGPT-Chinese-English-90k: A Bilingual Chinese-English Human-Machine Dialogue Dataset.* https://huggingface.co/datasets/shareAI/ShareGPT-Chinese-English-90k License: Apache-2.0.

[48] Noam Shazeer. 2019. Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150* (2019).

[49] Haiyang SHEN, Yue Li, Desong Meng, Dongqi Cai, Sheng Qi, Li Zhang, Mengwei Xu, and Yun Ma. 2025. ShortcutsBench: A Large-Scale Real-world Benchmark for API-based Agents. In *The Thirteenth International Conference on Learning Representations*.

[50] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. 2023. FlexGen: high-throughput generative inference of large language models with a single GPU. In *Proceedings of the 40th International Conference on Machine Learning*. 31094–31116.

[51] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. 2024. Llumnix: dynamic scheduling for large language model serving. In *Proceedings of the 18th USENIX Conference on Operating Systems Design and Implementation*. 173–191.

[52] Ruslan Svirschevski, Avner May, Zhuoming Chen, Beidi Chen, Zhihao Jia, and Max Ryabinin. 2024. SpecExec: Massively Parallel Speculative Decoding For Interactive LLM Inference on Consumer Devices. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.

[53] MLC team. 2023. *MLC-LLM.* https://github.com/mlc-ai/mlc-llm

[54] Chunlin Tian, Xinpeng Qin, Kahou Tam, Li Li, Zijian Wang, Yuanzhe Zhao, Minglei Zhang, and Chengzhong Xu. 2025. CLONE: customizing LLMs for efficient latency-aware inference at the edge. In *Proceedings of the 2025 USENIX Conference on Usenix Annual Technical Conference*. 563–585.

[55] Mircea Trofin, Yundi Qian, Eugene Brevdo, Zinan Lin, Krzysztof Choromanski, and David Li. 2021. Mlgo: a machine learning guided compiler optimizations framework. *arXiv preprint arXiv:2101.04808* (2021).

[56] Dana Van Aken, Andrew Pavlo, Geoffrey J. Gordon, and Bohan Zhang. 2017. Automatic Database Management System Tuning Through Large-scale Machine Learning. In *Proceedings of the 2017 ACM International Conference on Management of Data*. 1009–1024.

[57] Kuan Wang, Zhijian Liu, Yujun Lin, Ji Lin, and Song Han. 2019. Haq: Hardware-aware automated quantization with mixed precision. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 8612–8620.

[58] Wenxiao Wang, Wei Chen, Yicong Luo, Yongliu Long, Zhengkai Lin, Liye Zhang, Binbin Lin, Deng Cai, and Xiaofei He. 2024. Model compression and efficient inference for large language models: A survey. *arXiv preprint arXiv:2402.09748* (2024).

[59] Bingyang Wu, Ruidong Zhu, Zili Zhang, Peng Sun, Xuanzhe Liu, and Xin Jin. 2024. dLoRA: Dynamically Orchestrating Requests and Adapters for LoRA LLM Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. 911–927.

[60] Bowei Xi, Zhen Liu, Mukund Raghavachari, Cathy H. Xia, and Li Zhang. 2004. A smart hill-climbing algorithm for application server configuration. In *Proceedings of the 13th International Conference on World Wide Web*. 287–296.

[61] Heming Xia, Zhe Yang, Qingxiu Dong, Peiyi Wang, Yongqi Li, Tao Ge, Tianyu Liu, Wenjie Li, and Zhifang Sui. 2024. Unlocking efficiency in large language model inference: A comprehensive survey of speculative decoding. *arXiv preprint arXiv:2401.07851* (2024).

[62] Weikai Xie, Li Zhang, Shihe Wang, Rongjie Yi, and Mengwei Xu. 2024. Droidcall: A dataset for llm-powered android intent invocation. *arXiv preprint arXiv:2412.00402* (2024).

[63] Mengwei Xu, Dongqi Cai, Wangsong Yin, Shangguang Wang, Xin Jin, and Xuanzhe Liu. 2025. Resource-efficient Algorithms and Systems of Foundation Models: A Survey. *ACM Comput. Surv.* (2025).

[64] Rongjie Yi, Ting Cao, Ao Zhou, Xiao Ma, Shangguang Wang, and Mengwei Xu. 2023. Boosting DNN Cold Inference on Edge Devices. In *Proceedings of the 21st Annual International Conference on Mobile Systems, Applications and Services*. 516–529.

[65] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. 521–538.

[66] Haolin Zhang and Jeff Huang. 2025. Challenging GPU Dominance: When CPUs Outperform for On-Device LLM Inference. *arXiv preprint arXiv:2505.06461* (2025).

[67] Ji Zhang, Yu Liu, Ke Zhou, Guoliang Li, Zhili Xiao, Bin Cheng, Jiashu Xing, Yangtao Wang, Tianheng Cheng, Li Liu, Minwei Ran, and Zekang Li. 2019. An End-to-End Automatic Cloud Database Tuning System Using Deep Reinforcement Learning. In *Proceedings of the 2019 International Conference on Management of Data*. 415–432.

[68] Xinlu Zhang, Zhiyu Zoey Chen, Xi Ye, Xianjun Yang, Lichang Chen, William Yang Wang, and Linda Ruth Petzold. 2025. Unveiling the impact of coding data instruction fine-tuning on large language models reasoning. In *Proceedings of the AAAI Conference on Artificial Intelligence*. 25949–25957.

[69] Xiaoyuan Zhang, Xi Lin, Bo Xue, Yifan Chen, and Qingfu Zhang. 2023. Hypervolume maximization: A geometric view of pareto set learning. *Advances in Neural Information Processing Systems*, 38902–38929.

[70] Xinyi Zhang, Hong Wu, Zhuo Chang, Shuowei Jin, Jian Tan, Feifei Li, Tieying Zhang, and Bin Cui. 2021. Restune: Resource oriented tuning boosted by meta-learning for cloud databases. In *Proceedings of the 2021 international conference on management of data*. 2102–2114.

[71] Xinyi Zhang, Hong Wu, Yang Li, Jian Tan, Feifei Li, and Bin Cui. 2022. Towards Dynamic and Safe Configuration Tuning for Cloud Databases. In *Proceedings of the 2022 International Conference on Management of Data*. 631–645.

[72] Zongpu Zhang, Pranab Dash, Y Charlie Hu, Qiang Xu, Jian Li, and Haibing Guan. 2025. Dissecting the Impact of Mobile DVFS Governors on LLM Inference Performance and Energy Efficiency. *arXiv preprint arXiv:2507.02135* (2025).

[73] Yilong Zhao, Chien-Yu Lin, Kan Zhu, Zihao Ye, Lequn Chen, Size Zheng, Luis Ceze, Arvind Krishnamurthy, Tianqi Chen, and Baris Kasikci. 2024. Atom: Low-Bit Quantization for Efficient and Accurate LLM Serving. In *Proceedings of Machine Learning and Systems*. 196–209.

[74] Lianmin Zheng, Chengfan Jia, Minmin Sun, Zhao Wu, Cody Hao Yu, Ameer Haj-Ali, Yida Wang, Jun Yang, Danyang Zhuo, Koushik Sen, Joseph E. Gonzalez, and Ion Stoica. 2020. Ansor: generating high-performance tensor programs for deep learning. In *Proceedings of the 14th USENIX Conference on Operating Systems Design and Implementation*. 863–879.

[75] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Livia Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. 2024. Sglang: Efficient execution of structured language model programs. *Advances in neural information processing systems* (2024), 62557–62583.

[76] Zhen Zheng, Xin Ji, Taosong Fang, Fanghao Zhou, Chuanjie Liu, and Gang Peng. 2024. Batchllm: Optimizing large batched llm inference with global prefix sharing and throughput-oriented token batching. *arXiv preprint arXiv:2412.03594* (2024).

[77] Yuqing Zhu, Jianxun Liu, Mengying Guo, Yungang Bao, Wenlong Ma, Zhuoyue Liu, Kunpeng Song, and Yingchun Yang. 2017. BestConfig: tapping the performance potential of systems via automatic configuration tuning. In *Proceedings of the 2017 Symposium on Cloud Computing*. 338–350.

A Appendix

A.1 Tunable Parameters in Inference Engines

As discussed in Section 2.1, modern on-device inference engines expose a rich set of tunable parameters across different functional layers. These parameters govern how models are prepared, executed, scheduled, and served, and together they determine the overall inference efficiency in terms of throughput, latency, and resource utilization. Following recent studies on inference engine architecture and optimization [15, 32, 42, 58], we categorize these parameters into four hierarchical levels according to their operational scope: *model preparation*, *execution*, *system management*, and *serving*.

Table 2 summarizes representative parameters at each level. Model-preparation parameters control static model transformations such as quantization or pruning, which trade accuracy for efficiency and are typically applied offline. Execution-level parameters affect kernel selection, operator fusion, and memory layout during inference. System-level parameters configure hardware resource usage, including the number of threads, memory pooling, and scheduling strategies. Finally, serving-level parameters relate to request handling, concurrency, and endpoint configuration in multi-session deployment scenarios.

In our experiments, we focus on optimizing system-level parameters that directly influence runtime performance while keeping model quality and service semantics unchanged.

Table 2: Hierarchy of tunable parameters in on-device LLM inference engines.

Level	Description	Example
Model prep.	Transforms model artifacts; trades accuracy–efficiency (often offline).	`quantization`, `sparsity`
Execution	Chooses kernels/graphs and memory layout for operators.	`attention_impl`, `kv_cache_dtype`
System	Manages device resources and scheduling on the target.	`threads`, `parallel`, `mem_pool`
Serving	Controls admission, concurrency, and endpoints.	`concurrency`, `max_queue_delay`, `host`, `port`

A.2 Meta-feature Definitions for Knowledge Transfer

To support knowledge transfer across tuning tasks, LaTune represents each task by a compact *meta-feature vector* that encodes salient properties of the model, hardware, and workload. These meta-features serve as descriptors that enable similarity measurement between tasks and guide the warm-start process described in Section 4.3. Table 3 lists all feature categories and their definitions.

A.3 Real-World Validation

In addition to synthetic loads, we evaluate LaTune under real-world background activities on an NVIDIA RTX 3060 with the *Phi-Mini-MoE-q8* model. We introduce three distinct contention scenarios:

- **Video Playback:** 4 K video streaming in the background.
- **Browser Gaming:** Running a WebGL-based browser game.

Table 3: Meta-feature definitions for task representation in LaTune.

Category	Feature	Description
Model	n_params_billion	Model parameters (billions).
	n_layer	Transformer layers.
	n_embd	Hidden embedding dimension.
	n_head	Attention heads.
	n_head_kv	Key-value heads (grouped attention).
	n_ff	Feed-forward dimension.
	context_len_train	Training-time context length.
	is_moe	Uses Mixture-of-Experts.
	quant_file_type	Quantization type (Q4, Q8, etc.).
	file_size_GiB	Model file size (GiB).
	bpw	Bits per weight.
Hardware	gpu_mem_GB	GPU memory (GB).
	cpu_threads	CPU threads for inference.
	gpu_vendor	GPU vendor (e.g., NVIDIA).
Workload	task_type	Task type (`chat`, `summary`).
	avg_input_len	Avg. input token length.

- **Multi-Inference:** Running two concurrent LLM serving instances.

As shown in Table 4, LaTune maintains high throughput and stability across all scenarios. Notably, in high-contention cases like Gaming and Multi-Inference, default configurations and Genetic Algorithms (GA) fail (denoted by ×) due to resource exhaustion, whereas LaTune successfully adapts to feasible configurations.

Table 4: Throughput (TPS) under Real-World Contention Scenarios (RTX 3060).

Scenario	LaTune	Default	GA	ResTune	SCOOT
Video	**93.06**	69.45	85.86	85.85	86.72
Game	**76.27**	×	×	70.78	70.57
Multi-Inf	**49.62**	×	×	×	×

A.4 Discussion and Limitations

Generalizability to Unseen Environments. LaTune may face challenges with unseen hardware architectures where learned meta-features are less effective. In such cases, the system degrades gracefully to a "cold-start" mode, ensuring functionality by accumulating data from scratch without the warm-start advantage.

Static Parameter Selection. Our offline analysis assumes parameter sensitivity remains stable. Significant changes to the inference engine or model architecture require re-triggering this analysis, incurring a one-time cost of approximately 100 observations to update the importance rankings.

Dependence on Ranking Stability. Our two-stage design relies on the consistency of configuration rankings under dynamic loads. Extreme conditions, such as severe thermal throttling, may temporarily violate this assumption, suggesting a potential area for future online recalibration mechanisms.